

Semana 11 — Navigation, Behavior Trees (LimboAI), Blackboards e Combate Simples

Semana 11

Navigation, Behavior Trees (LimboAI), Blackboards e Combate Simples

Disciplina: Tendências de Motores de Jogos (IN46A)

Curso: Tecnologia em Jogos Digitais — IFMS Campus Dourados

Unidade III — Resolver Problemas (Semanas 8–11)

Projeto: Vertical Slice *O Templo Esquecido*

Encerramento de Módulo — Playtest coletivo e Showcase

IFMS · Tecnologia em Jogos Digitais · Tendências de Motores de Jogos

Objetivos da Semana

- Compreender Navigation (`NavigationRegion3D` / `NavigationAgent3D`) como base universal de deslocamento autônomo de agentes, independente de engine
- Compreender Behavior Tree e Blackboard como estrutura de decisão e memória compartilhada de um agente não-jogador, via addon LimboAI
- Implementar um combate simples que reutiliza o `HealthComponent` já existente, sem duplicar lógica entre `Player` e `Enemy`
- Propor e implementar, com solução própria, um comportamento autônomo para o `Enemy` do próprio projeto
- Consolidar e apresentar o Vertical Slice jogável do Módulo 3 em Playtest coletivo e Showcase

Semana 11 — Navigation, Behavior Trees (LimboAI), Blackboards e Combate Simples

ENCONTRO 1

Navigation: Ensinar um Agente a se Deslocar

IFMS · Tecnologia em Jogos Digitais · Tendências de Motores de Jogos

Agenda do Encontro 1

- Revisão do Encontro 2 da Semana 10 (Interaction System ampliado + inventário) (15 min)
- Introdução: por que um agente não-jogador precisa de um sistema dedicado de deslocamento (20 min)
- Demonstração: bake guiado de `NavigationRegion3D` e configuração de um `NavigationAgent3D` (30 min)
- Laboratório: cada grupo faz o bake da navegação no próprio nível e movimenta um agente de teste (55 min)
- Feedback e fechamento (15 min)



O Player usa move_and_slide com input do teclado. Como o Enemy se move sem teclado nenhum?

Pense no que muda quando o "input" passa a ser uma decisão da própria engine.

O Problema: Deslocamento sem Input Direto

- Todo agente não-jogador que precisa se mover enfrenta o mesmo problema estrutural, em qualquer engine
- Calcular uma rota sobre uma superfície navegável e segui-la evitando obstáculos
- Essa lógica de baixo nível não pode ser reescrita a cada novo tipo de agente

NavigationRegion3D e NavigationAgent3D

- **NavigationRegion3D** — a área navegável, calculada previamente (bake) sobre a geometria do nível
- **NavigationAgent3D** — componente de deslocamento: dado um ponto-alvo, calcula e segue a rota dentro dessa região
- Deliberadamente introduzido antes de qualquer decisão de IA: primeiro o agente se move de A a B, depois — no Encontro 2 — aprende a decidir para onde ir

Navegação no Godot × Unity

Godot

- `NavigationRegion3D` — malha navegável, bake no editor sobre a geometria do nível
- `NavigationAgent3D` — calcula e segue rota até um ponto-alvo

Unity

- `NavMesh` — malha navegável, gerada por bake (historicamente via NavMesh Baking / AI Navigation)
- `NavMeshAgent` — calcula e segue rota até um destino

Demonstração — Bake e Movimentação Guiados

O que será construído:

- Um `NavigationRegion3D` calculado sobre a geometria existente do nível
- Um `NavigationAgent3D`, em cena de teste, movendo-se até um ponto-alvo fixo

Por quê: dar a cada grupo a base direta para o `Enemy` do Encontro 2, sem qualquer lógica de decisão ainda.

Imagem sugerida

Objetivo: mostrar a malha de navegação calculada sobre a geometria do nível e a rota seguida por um agente de teste.

Enquadramento: vista superior do nível do Vertical Slice, malha de navegação sobreposta em destaque.

*Elementos presentes: **NavigationRegion3D** (malha translúcida sobre o chão navegável), um agente de teste (cubo simples) com uma linha pontilhada indicando a rota até um ponto-alvo.*

Destaque visual: contraste entre a área navegável (colorida) e obstáculos fora da malha (cinza).

Legenda sugerida: "O NavigationAgent3D segue a rota calculada pelo NavigationServer sobre a malha do NavigationRegion3D."

Laboratório — Navegação Funcional

Cada grupo, no próprio nível:

1. Faz o bake de um `NavigationRegion3D` sobre a geometria existente
2. Configura um `NavigationAgent3D` em uma cena de agente de teste
3. Movimenta esse agente até um ponto-alvo fixo, sem colidir com obstáculos

Fechamento — Encontro 1

- `NavigationRegion3D` calculado sobre o nível de cada grupo
- Agente de teste deslocando-se autonomamente até um ponto-alvo, sem colidir com obstáculos
- Nenhuma alteração nos sistemas de gameplay já funcionais da Semana 10
- Próximo passo: Behavior Tree, Blackboard e combate simples, no Encontro 2

Semana 11 — Navigation, Behavior Trees (LimboAI), Blackboards e Combate Simples

ENCONTRO 2

Behavior Tree, Blackboard e Combate Simples

IFMS · Tecnologia em Jogos Digitais · Tendências de Motores de Jogos

Agenda do Encontro 2

- Revisão do Encontro 1 (navegação funcional do agente de teste) (15 min)
- Demonstração: Behavior Tree/Blackboard (LimboAI) — patrulha/perseguição — e combate simples (30 min)
- Apresentação do desafio: comportamento autônomo adicional do **Enemy** (15 min)
- Laboratório/Desafio: Behavior Tree, combate simples e comportamento escolhido (45 min)
- Playtest coletivo e Showcase — encerramento do Módulo 3 (30 min)

Montando o `Enemy` — Mesmo Rig, Outra Skin

- `Enemy` (`CharacterBody3D`) reutiliza o mesmo modelo do Kenney Mini Characters já usado pelo Player desde a Semana 8
- Skin diferente, mesmo `AnimationPlayer` e mesmas animações (idle, walking, running) — nenhum asset novo é importado
- Reforça o mesmo princípio de reutilização do `HealthComponent` : um pacote, dois personagens



O agente já sabe se mover até um ponto. Quem decide qual ponto?

Pense na diferença entre "como se mover" (Encontro 1) e "o que fazer" (agora).

O Problema: Decidir, Não Apenas Mover

- O Encontro 1 resolveu "como" o `Enemy` se move
- Falta resolver "o que" ele decide fazer com esse deslocamento — patrulhar, perseguir, atacar
- Essa decisão não deve criar um sistema de movimentação paralelo ao `NavigationAgent3D` já configurado

Behavior Tree e Blackboard (LimboAI)

- **Behavior Tree** — estrutura de decisão hierárquica: sequências (passos em ordem), seletores (alternativas até uma funcionar), folhas (ações/condições)
- **Blackboard** — memória compartilhada entre os nós da árvore (ex.: última posição conhecida do `Player`)
- Nenhum dos dois é nativo do Godot — resolvidos pelo addon LimboAI

Decisão Autônoma no Godot × Unity

Godot

- LimboAI (addon, não nativo) — `BTPlayer`, Blackboard
- Nós de sequência, seletor e folha; memória compartilhada no Blackboard
- Tasks customizadas (`BTAction` / `BTCondition`) exigem GDScript — fora do alcance do Orchestrator

Unity

- Behavior Designer ou NodeCanvas (assets de terceiros, não nativos)
- Mesma lógica de nós; memória compartilhada por variáveis do asset ou `ScriptableObject` de estado

Demonstração — Patrulha, Perseguição e Combate

O que será construído:

- Behavior Tree simples de patrulha (deslocamento entre pontos via `NavigationAgent3D`) e perseguição (mudança de alvo ao detectar o `Player`) — tasks em GDScript, exigência do LimboAI
- Combate simples: `Area3D` / `RayCast3D` do `Player` chamando `apply_damage` no `HealthComponent` do `Enemy`, via Orchestrator ou GDScript

Por quê: fixar o padrão guiado antes de abrir o desafio do comportamento autônomo adicional.

Imagem sugerida

*Objetivo: mostrar a Behavior Tree do **Enemy** decidindo entre patrulha e perseguição, usando o Blackboard como memória compartilhada.*

Enquadramento: diagrama de árvore, seletor no topo com dois ramos (Patrulhar / Perseguir).

*Elementos presentes: nó seletor no topo; ramo "Patrulhar" com sequência de pontos; ramo "Perseguir" lendo a posição do **Player** a partir do Blackboard; folha final acionando o **NavigationAgent3D**.*

Destaque visual: o ramo ativo no momento (Perseguir) em cor sólida, o ramo inativo (Patrulhar) esmaecido.

Legenda sugerida: "A Behavior Tree decide; o NavigationAgent3D, já configurado no Encontro 1, executa o deslocamento."

Arquitetura — Decisão, Deslocamento e Combate

- Behavior Tree/Blackboard (decisão) → `NavigationAgent3D` (deslocamento, Encontro 1) → `Enemy` se move
- `Area3D` / `RayCast3D` do `Player` → `apply_damage` → `HealthComponent` do `Enemy` (mesmo Component da Semana 8)
- Nenhum sistema anterior é reescrito: cada camada reutiliza a anterior



Desafio — Comportamento Autônomo do Enemy

Proponha e implemente um comportamento autônomo adicional para o **Enemy** do seu projeto — **patrulha, alerta, fuga** ou **interação com o jogador** — com liberdade de solução dentro da estrutura de Behavior Tree/Blackboard já fundamentada.

OBJETIVOS

Entrega: Vertical Slice jogável (encerramento do Módulo 3) — Playtest coletivo (Rubrica 5) e Showcase (Rubrica 6).

Boas Práticas — Camadas Separadas

- A Behavior Tree decide; o `NavigationAgent3D` executa o deslocamento — nunca recalcular posição manualmente dentro da árvore
- O `HealthComponent` é o mesmo para `Player` e `Enemy` — nunca duplicar lógica de vida/dano
- Manter a Behavior Tree simples: patrulha/perseguição básicas resolvem o escopo do combate simples do projeto

Playtest Coletivo e Showcase — Rubricas 5 e 6

- **Playtest** (Rubrica 5) — experiência efetiva de jogo, funcionamento, usabilidade e clareza para quem joga o Vertical Slice de cada grupo
- **Showcase** (Rubrica 6) — apresentação do projeto e capacidade de justificar decisões de arquitetura diante da turma
- Encerramento formal da Unidade III (Módulo 3 — Resolver Problemas)

Fechamento — Encontro 2

- Behavior Tree/Blackboard (LimboAI) decidindo entre patrulha e perseguição, reutilizando o `NavigationAgent3D` do Encontro 1
- Combate simples funcional, reutilizando o `HealthComponent` da Semana 8 sem duplicação
- Comportamento autônomo adicional proposto e implementado com solução própria
- Vertical Slice avaliado em Playtest coletivo e apresentado em Showcase

Resultado Esperado da Semana

- `Enemy` deslocando-se autonomamente por um `NavigationRegion3D`
- Comportamento decidido via Behavior Tree/Blackboard (LimboAI), incluindo um comportamento adicional proposto com solução própria
- Combate simples trocando dano entre `Player` e `Enemy`, reutilizando o `HealthComponent` sem duplicação de lógica
- Vertical Slice jogável completo — animação, HUD, inventário, interação, IA e combate — avaliado em Playtest e apresentado em Showcase

Checklist da Semana

- [] `NavigationRegion3D` calculado sobre o nível, sem áreas inacessíveis
- [] `Enemy` deslocando-se via `NavigationAgent3D`
- [] Behavior Tree/Blackboard (LimboAI) decidindo entre patrulha e perseguição
- [] Combate simples (`Area3D` / `RayCast3D` → `apply_damage`) reutilizando o `HealthComponent` sem duplicação
- [] Comportamento autônomo adicional implementado com solução própria
- [] Playtest coletivo e Showcase (Rubricas 5 e 6) concluídos

Próximos Passos — Semana 12

O Vertical Slice jogável consolidado nesta semana é a base do Módulo 4 (Unidade IV — Produzir como um Pequeno Estúdio), que inicia na Semana 12 com o refinamento de Materials, Material Overrides e Foliage (`MultiMeshInstance3D`) — nenhum sistema novo de gameplay é introduzido; o foco passa a ser polimento técnico e produção em escala de estúdio, com autonomia alta e o professor atuando como diretor técnico.

Leitura recomendada: Godot Docs — Navigation, Physics (Area3D, RayCast3D); LimboAI (github.com/limbonaut/limboai); Unity Manual (consulta comparativa) — NavMesh.